The observability dashboard

Every panel, chart, button and number on the Crickbot dashboard: what it shows, how it's calculated, and what to say when someone asks

http://127.0.0.1:8000/#ops · the Dashboard tab

Why it exists: three questions

Is it up?

Status banner, component list, service level, alerts

Is it fast?

Answer time p95, LLM queue, feed lag, traces

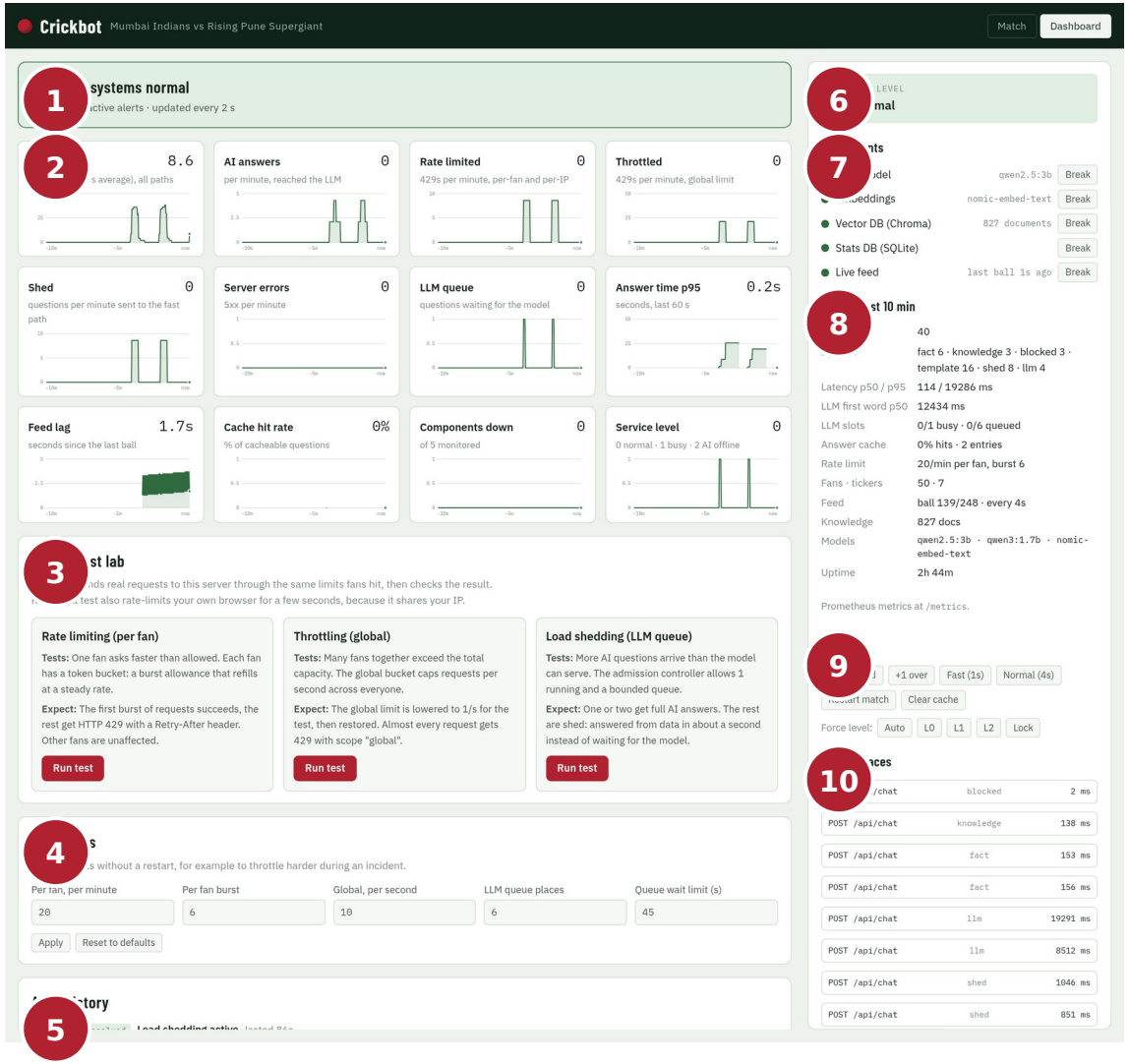
Is it coping?

Rate limited, throttled, shed, server errors, test lab

The three kinds of signal

Metrics	**numbers over time**	"How many answers per minute?" → the 12 charts, /metrics
Logs	**one line per event**	"What exactly happened at 19:13?" → backend/var/app.log
Traces	**the steps of one request**	"Why was this answer slow?" → Recent traces

Map of the dashboard



1 **Status banner**

overall: normal, degraded or down

2 **12 live charts**

the last 10 minutes

3 **Traffic test lab**

rate limit, throttle, shed tests

4 **Live limits**

change limits without restarting

5 **Alert history**

every alert fired and resolved

6 **Service level**

L0 normal · L1 busy · L2 AI offline

7 **Components**

5 parts, each with Break

8 **Traffic, last 10 min**

answers, latency, slots, cache…

9 **Operator**

feed, match, cache, force level

10 **Traces / errors**

per-answer timings; last failures

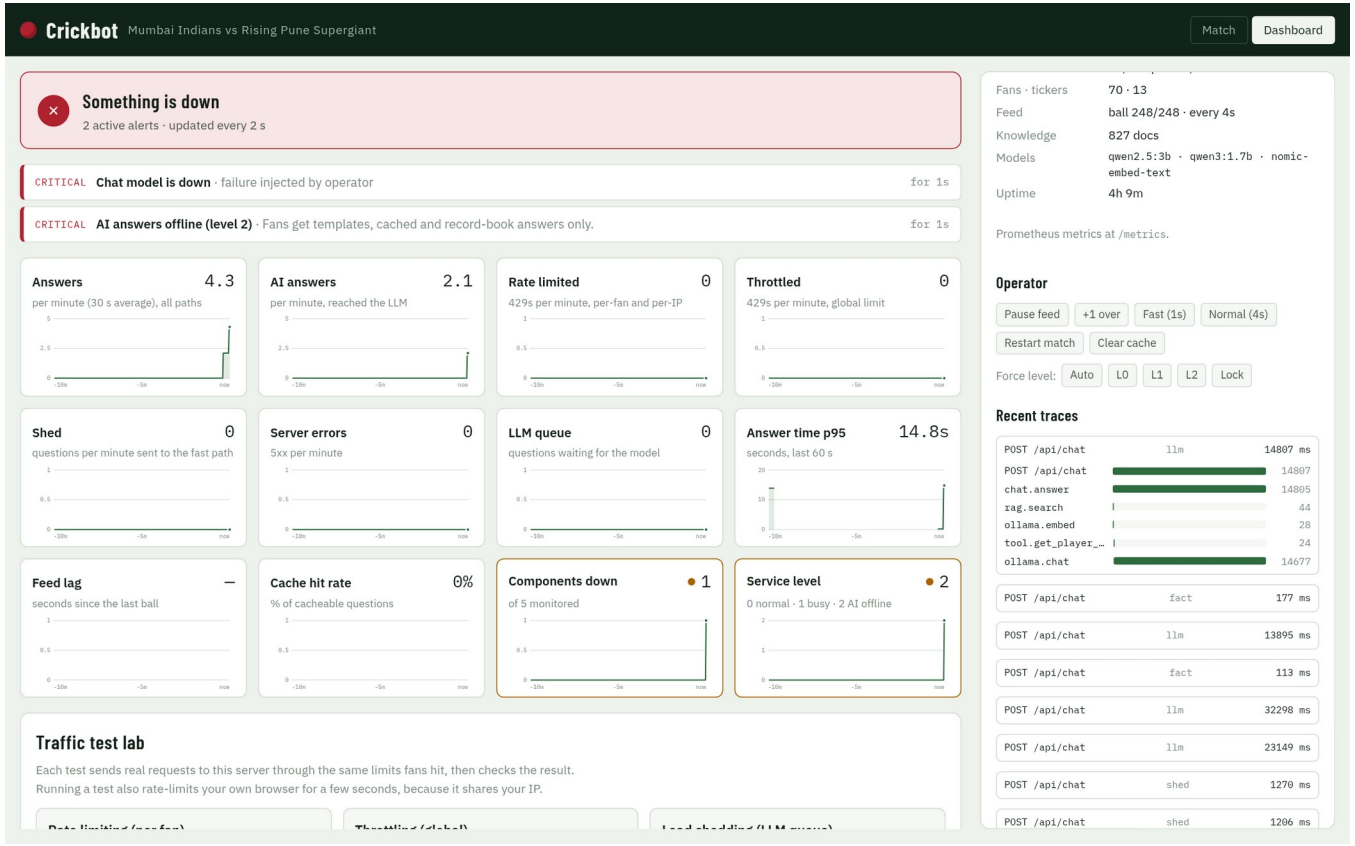
Left column: what's happening. Right column: what it's made of, and the controls.

How the numbers get to the screen



Runs in the background	How often	What it does
Monitor sampler	every 2 s	rates, gauges, alert rules
Health probe	every 5 s	real check of Ollama, Chroma, SQLite; closes breakers when they recover
Dashboard charts + alerts	every 2 s	browser fetches the time series and alerts
Right panel (status)	every 3 s	browser fetches /api/ops/status
Traces + errors (unlocked)	every 4 s	browser fetches the admin endpoints

Status banner and alerts



Three states

Normal — no alerts, or only info alerts (e.g. fans rate limited)

Degraded — a warning alert: shedding, throttling, busy, slow, feed delayed, queue full

Down — a component is down or the server returned 5xx errors

Each alert shows: severity (critical / warning / info), title, the reason, and how long it has been active ("for 1s").

In the screenshot: Chat model broken on purpose → "Chat model is down" and "AI answers offline (level 2)", and the Components down and Service level charts get amber borders.

The 10 alert rules

Alert	Severity	Fires when	Clears when
A component is down	**critical**	any of the 5 parts is down (breaker open or Break pressed)	the part works again (probe, every 5 s)
AI answers offline (level 2)	**critical**	the chat model is unavailable	the model is back
Busy mode (level 1)	**warning**	the AI queue is at least half full	the queue drains
Server errors	**critical**	any HTTP 5xx in the last 60 s	60 s with no 5xx
Live feed delayed	**warning**	no ball for more than 20 s while the match runs	the next ball arrives
Load shedding active	**warning**	any question shed in the last 60 s	60 quiet seconds
Global throttle hit	**warning**	any global 429 in the last 60 s	60 quiet seconds
Fans rate limited	**info**	any per-fan or per-IP 429 in the last 60 s	60 quiet seconds
Slow answers	**warning**	p95 answer time above 30 s in the last 60 s	p95 back under 30 s
LLM queue full	**warning**	all waiting places are taken	a place frees up

Break and Restore, second by second

0 s	**Break**	the part is added to a "chaos" list; every call to it now fails on purpose. Nothing is damaged.
0–2 s	**Breaker opens**	after 3 failures in a row the app stops calling that part (only if something used it)
≤ 2 s	**Alert fires**	the monitor's next check sees it down: red banner, "ALERT fired" in the log
meanwhile	**App degrades**	chat model down → data answers only (L2); vector DB down → stats still work
Restore	**Stop the simulation**	the part is removed from the chaos list
≤ 5 s	**Probe closes the breaker**	a real health check succeeds, so the breaker closes
≤ 2 s	**Alert resolves**	banner green; "resolved" in Alert history with how long it lasted

The 12 charts (1 of 2)

Chart	What it shows	How it's calculated	Normal	Amber border when
Answers	all answers per minute	counter of finished answers, 30 s average	rises with traffic	never
AI answers	answers that reached the LLM	same, LLM path only	low: most answers skip the AI	never
Rate limited	per-fan + per-IP 429s per minute	rate-limit counter	0	above 0
Throttled	global 429s per minute	rate-limit counter, global scope	0	above 0
Shed	questions sent to the fast path	load-shed counter	0	above 0
Server errors	HTTP 5xx per minute	HTTP counter, status 5xx	0	above 0

The 12 charts (2 of 2)

Chart	What it shows	How it's calculated	Normal	Amber border when
LLM queue	questions waiting for the AI	live gauge	0	above 0
Answer time p95	95% of answers were faster than this	last 60 s of answers	~0.2 s; spikes for AI answers	above 30 s
Feed lag	seconds since the last ball	live gauge	0–4 s sawtooth	above 20 s
Cache hit rate	share of repeat questions served from cache	hits ÷ (hits + misses) per 2 s	often 0%: questions differ	never
Components down	how many of the 5 parts are down	health registry	0	above 0
Service level	0 normal · 1 busy · 2 AI offline	degrade level	0	above 0

How to read a chart

Big number, top right

the latest value

Hover

a dotted line and the exact value and time

Amber border + dot

this chart needs attention now

Flat line at 0

nothing happening: good for 429s, sheds and errors

Gap in the line

no data, e.g. p95 when nobody asked anything in the last 60 s

Spike, then a slow fade

a burst, e.g. a traffic test; the 30 s average makes it fade

Sawtooth (Feed lag)

climbs 0→4 s, drops when each ball arrives: healthy

X-axis

−10 min on the left, now on the right

Service level and components

Service level

L0 **Normal**

everything on

L1 **Busy**

AI queue at least half full → thinking off, shorter answers

L2 **AI offline**

chat model unavailable → templates, cache and data answers only

Chosen automatically; Force level can override it for a demo.

Component	What it is	Note shown	If it's down
Chat model	Ollama qwen2.5:3b	model name	L2: data answers only
Embeddings	nomic-embed-text	model name	no knowledge search
Vector DB	Chroma	827 documents	no knowledge search
Stats DB	SQLite	—	no stats lookups
Live feed	ball replay	last ball Ns ago	score freezes

Dot: green up, red down. **Breaker states:** closed (normal), open (failing, not called), half-open (trying one test call after 20 s).

"Traffic, last 10 min": every line explained

Line	Meaning	Example
Answers	answers finished in the last 10 minutes	`24`
By path	how they were answered (next slide)	`fact 6 · template 8 · llm 2`
Latency p50 / p95	half / 95% of answers were faster than	`132 / 8787 ms`
LLM first word p50	typical time until the AI's first word	`15230 ms`
LLM slots	AI answers running / max · waiting / max	`0/1 busy · 0/6 queued`
Answer cache	repeat questions served instantly · entries kept (2 min)	`0% hits · 8 entries`
Rate limit	the current per-fan limit	`20/min per fan, burst 6`
Fans · tickers	chat sessions that asked something · open live-score streams	`40 · 7`
Feed	match progress and speed	`ball 108/248 · every 4s`
Knowledge	documents in Chroma	`827 docs`
Models	chat · thinking · embedding	`qwen2.5:3b · qwen3:1.7b · nomic-embed-text`
Uptime	time since the app started	`2h 42m`

"By path": the 12 ways a question can be answered

`template` — 0–1 ms
live score, batting, chase

`cache` — 1 ms
a repeat question

`fact` — ~150 ms
composed from the stats DB

`knowledge` — ~130 ms
rule book text

`llm` — 7–17 s
written by the AI model

`coalesced` — as llm
shared one in-flight AI answer

`shed` — ~1 s
queue full → quick data answer

`degraded` — <1 s
AI down → data answer

`scope` — ~2 ms
outside 2008–2017 / World Cup

`off_topic` — ~120 ms
not cricket

`blocked` — 0 ms
attack, betting, abuse, too long

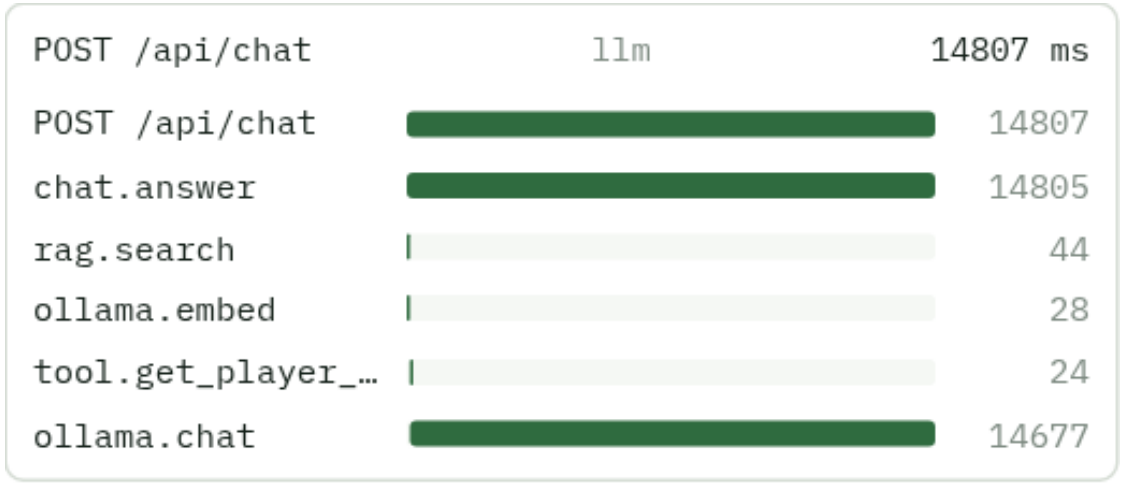
`clarify` — 0 ms
"and", "his?": asks for more

Operator buttons

Button	What it does	Use it to…
Pause feed / Resume feed	stops / restarts the ball-by-ball replay	show the "Live feed delayed" alert (after 20 s)
+1 over	applies the next 6 deliveries at once	move the match on quickly
Fast (1s) / Normal (4s)	one ball every 1 s or every 4 s	speed up the demo
Restart match	replays the final from ball 1 and clears the answer cache	start the demo fresh
Clear cache	forgets cached answers	force fresh answers
Force level: Auto / L0 / L1 / L2	overrides the service level	show L2 behaviour without breaking anything
Break / Restore (Components)	simulates a failure of one part	show graceful degradation
Lock	forgets the admin token in this tab	hand the screen to someone else

All of these need the admin token. Without it the dashboard is read-only.

Recent traces and recent errors



A real AI answer: 14,807 ms in total. Search 44 ms, embedding 28 ms, player-stats tool 24 ms, LLM 14,677 ms: 99% of the time is the model.

Span	Meaning
`POST /api/chat`	the whole web request
`chat.answer`	the agent's work for one question
`rag.search`	vector + keyword search in Chroma
`ollama.embed`	turning the question into 768 numbers
`tool.<name>`	one database lookup, e.g. get_player_stats
`ollama.chat`	the AI model writing (tokens in/out recorded)

Traces: OpenTelemetry spans for the last 50 chat answers; click one to see its waterfall. Bars show when each step started and how long it took. **Recent errors:** the last 100 failures recorded by the circuit breakers, with time, component and message.

Traffic test lab: what each test really does

Test	What it sends	PASS means	Afterwards
Rate limiting	One fan, 10 quick questions	6 allowed, then 429 (scope: session, Retry-After 3 s)	nothing to restore
Throttling	4 fans, 12 questions at once; global limit lowered to 1/s	2 allowed, 10 throttled (scope: global)	global limit restored to 10/s
Load shedding	6 AI questions at once; queue cut to 1 place	1–2 AI answers, 4 shed in about 1 s, 0 errors	queue restored to 6

Real requests: each test calls this server over HTTP through the same middleware, limits and agent as fans.

One at a time: a second test while one runs gets "another test is already running".

Side effect: your own browser can get a 429 for a few seconds, because it shares the IP.

Alerts: the tests trigger shedding, throttling and rate-limit alerts; they clear after 60 quiet seconds.

Live limits: tuning without a restart

Field	Default	Allowed	What it controls
Per fan, per minute	20	1–10,000	steady questions per fan
Per fan burst	6	1–1,000	quick questions allowed at once (per IP: 3×)
Global, per second	10	above 0 – 10,000	total for everyone (burst: 2×)
LLM queue places	6	0–1,000	questions allowed to wait for the AI
Queue wait limit (s)	45	above 0 – 600	longest wait before a fast answer instead

Apply takes effect immediately; all limit counters restart full.

Reset to defaults goes back to the values in config.

Bad values (negative, NaN, Infinity) are rejected with a clear error.

Why it's useful: during an incident, throttle harder in seconds instead of redeploying.

Behind the dashboard: endpoints and logs

Endpoint	Who can call it	Returns
`/metrics`	anyone (localhost)	Prometheus format: 21 metrics, e.g. chat_questions_total, llm_tokens_total
`/health/live`	anyone	"ok" if the process is running
`/health/ready`	anyone	each component's status and last error
`/api/ops/status`	anyone	the right panel's numbers
`/api/ops/timeseries`	anyone	the chart data, last 10 minutes
`/api/ops/alerts`	anyone	active alerts and history
`/api/admin/…`	admin token only	traces, errors, chaos, feed, limits, cache, tests

```
{"ts": "…", "level": "WARNING", "logger": "alerts", "msg": "ALERT fired: Load shedding active", "request_id": "8893f6507fd6d4ca", …}
```

backend/var/app.log: one JSON line per event; emails and phone numbers redacted. Every response has an X-Request-ID header to find its lines.

Security of the dashboard

Read-only for everyone

charts, status and alerts need no login; the app only listens on 127.0.0.1

Controls need the admin token

checked in constant time; wrong token → 403 and the panel locks again

Token kept per tab

stored in sessionStorage: gone when the tab closes; Lock forgets it now

No secrets on screen

the token is never displayed; logs redact emails, phone numbers and bearer tokens

For a real deployment: put the dashboard behind company login and HTTPS, and give each operator their own account.

What this dashboard doesn't do (yet)

Limit	What it means	Production fix
Only 15 minutes of history, in memory	a restart clears charts and alert history	Prometheus + Grafana for long-term storage
One app process	it can't combine numbers from several copies	central metrics store (Prometheus)
Alerts only on screen and in the log	nobody gets woken up	Alertmanager → email, Slack or pager
Browser polls every 2–4 s	small delay; fine for one operator	push updates or Grafana live
Traces kept for 50 answers	older ones are dropped	send spans to Jaeger or Tempo (OTLP)

If they ask about the dashboard (1 of 2)

Is this real data or a mock?	Real. Every number comes from counters the app updates as it serves questions.
Why did that alert stay after the test?	Traffic alerts look at the last 60 seconds; they clear after a quiet minute.
Why is the cache hit rate 0%?	Questions differ, live ones are tied to the current ball, and answers expire after 2 minutes.
Why p95 and not the average?	An average hides the slow answers; p95 shows what the slowest 1 in 20 fans waited.
Why does Answers show a number when nobody's asking?	It's a 30-second average, so it fades after the last activity.
What does Break actually do?	Makes calls to that part fail on purpose. Nothing is damaged; Restore stops it.
How fast is a failure detected?	Within about 2 seconds. Recovery is noticed within about 7 seconds.
What if the dashboard itself breaks?	Fans aren't affected: it only reads status. The app keeps serving.

If they ask about the dashboard (2 of 2)

Who can press Break?	Only someone with the admin token from backend/.env.
Can this go into Grafana?	Yes: /metrics is Prometheus format, and traces can be sent via OpenTelemetry.
Why are there two levels of alert colour?	Red means something is down or erroring; amber means it's coping (shedding, throttling, slow).
How many things are monitored?	5 components, 12 charts, 10 alert rules, 21 Prometheus metrics, and traces for each answer.
Where are the logs?	backend/var/app.log, one JSON line per event; search by the X-Request-ID header.
Why is Feed lag a sawtooth?	It counts up between balls and drops to 0 when each ball arrives every 4 seconds.
What does Service level 1 mean?	The AI queue is at least half full: thinking turns off and answers get shorter to catch up.
How do you know the alerts work?	Automated tests check that an alert fires on failure and resolves on recovery, plus the 10 failure drills.

A 60-second dashboard tour

1	**Banner**	*"Green means all systems normal. It turns amber when the app is coping, red when something is down."*
2	**Charts**	*"The last 10 minutes: traffic, rejections, sheds, errors, the AI queue, answer time and feed lag."*
3	**Right panel**	*"Service level, the 5 components, and the last 10 minutes of traffic in numbers."*
4	**Break**	*"I'll break the chat model. Two seconds later: red banner, two critical alerts. The chat still answers stats."*
5	**Restore**	*"Restore… and within a few seconds it's green again, with the whole incident in Alert history."*
6	**Trace**	*"And this is one AI answer step by step: 99% of the time is the model, which is why most answers skip it."*